Cómputo Concurrente 2026
Práctica 3: Exclusión Mutua: Candados Clásicos y el Modelo de Memoria de Java (JMM)

Alumna: Montes de Oca Aguilar Yessica Vianney

1. El algoritmo de candado Peterson solo funciona para 2 hilos. Utiliza el algoritmo de candado Peterson para implementar un algoritmo de candado que funcione para 4 hilos.

   Respuesta:

   `PetersonID.java`:

   Versión del candado Peterson que recibe el id (0 o 1) como parámetro de `lock(int i)` y `unlock(int i)`, en vez de calcularlo internamente con `Thread.currentThread().getId() % 2` como en `Peterson.java` original. Esto es necesario porque, con un pool de más de 2 hilos, el id real del hilo del sistema operativo no garantiza mapear siempre a 0 o 1 de forma consistente entre distintos candados; si dos hilos distintos calculan el mismo id dentro del mismo candado, se rompe la exclusión mutua.

   `DoblePeterson.java`:

   Candado para 4 hilos armado como un árbol de torneo de tres candados `PetersonID`: uno para el par de hilos globales {0,1} (`left`), otro para el par {2,3} (`right`), y un tercero (`top`) donde compite el ganador de `left` contra el ganador de `right`. Un hilo con id global i primero adquiere su candado de hoja con id local i mód 2, y al ganarlo compite en `top` con id $\lfloor i/2 \rfloor$. Solo entra a la sección crítica quien gane ambos niveles.

   (a) Para probar que tu candado funciona, crea una implementación en donde utilices un `ExecutorService` para ejecutar 400 tareas, cada tarea debe aumentar en uno un objeto `Contador`. Utiliza tu candado para 4 hilos para tener consistencia en tu `Contador`.

   Respuesta:

   `ExecuteDoblePeterson.java`:

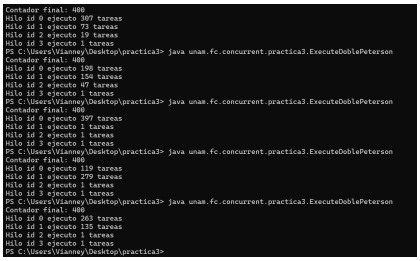
   Programa que reutiliza `CounterNaive` y ejecuta 400 tareas en un `ExecutorService` de 4 hilos (pool fijo). A cada uno de los 4 hilos reales del pool se le asigna un id fijo (0 a 3) mediante `ThreadLocal` la primera vez que toma una tarea, y ese id es el que usa durante toda

la ejecución para el candado `DoblePeterson`, evitando el problema de ids repetidos mencionado en la advertencia del enunciado.

(b) De alguna forma obtén el número de veces que los hilos aumentan el contador. ¿Cada uno realiza exactamente 100 tareas o hay algunos que realizan más?

Respuesta:

No, cada hilo no realiza exactamente 100 tareas; el reparto es muy desigual y cambia en cada ejecución. Se obtuvo:



En las 5 ejecuciones el patrón fue consistente: los hilos 0 y 1 (el par que compite en el candado `left`) se reparten casi toda la carga, mientras que los hilos 2 y 3 (par `right`) apenas alcanzan 1 tarea.

(c) ¿Consideras que la implementación cumple con **Justicia**? Justifica tu respuesta.

Respuesta:

No cumple con Justicia en el sentido de reparto equitativo. Peterson de 2 hilos sí garantiza *starvation-freedom* (nadie espera indefinidamente) dentro de cada candado hoja o del candado `top` por separado, y esa propiedad se preserva al componerlos en el árbol de torneo: ningún hilo se queda esperando para siempre. Sin embargo, starvation-freedom no es lo mismo que un orden FIFO ni que un reparto parejo.

Además del argumento anterior, los datos muestran un sesgo sistemático hacia el par `left` (hilos 0 y 1), no solo aleatorio. Esto se explica porque `Executors.newFixedThreadPool(4)` no crea los 4 hilos del pool simultáneamente, sino conforme se necesitan; los dos primeros hilos reales en arrancar (que por el orden de asignación del `ThreadLocal` terminan siendo los id 0 y 1) alcanzan a ejecutar varias tareas antes de que el pool termine de crear los otros dos hilos (id 2 y 3). Como Peterson es un candado "goloso"—un hilo que vuelve a pedirlo de inmediato normalmente le gana a uno que apenas está llegando a competir por primera vez—, esa ventaja inicial del par {0,1} se autoalimenta durante el resto de la ejecución. Esto confirma que el candado no corrige asimetrías de arranque entre hilos, otra forma en la que no se cumple Justicia en la práctica.

2. En base a la implementación de Bakery vista en la clase teoría, implementa Bakery para 4 hilos. Crea los arreglos `flag` y `label` de tamaño 4. Si consideras necesario, utiliza campos `volatile`.

   Respuesta:

   `Bakery4.java`:

   Implementación del algoritmo de Bakery de Lamport (versión de arreglos, no la de lista ligada del `Bakery.java`/`Nodo.java` de ejemplo) para 4 hilos, con arreglos `flag` y `label` de tamaño 4 declarados `volatile`. Cada hilo con id i levanta `flag[i]`, saca un número `label[i]` mayor a todos los números repartidos hasta ese momento, y espera mientras exista otro hilo k con `flag[k]` activo y con mejor prioridad que él (número más chico, o mismo número pero id más chico, usado como desempate).

   (a) Con ayuda de un `ExecutorService` ejecuta 400 tareas, cada tarea debe aumentar en uno un objeto `Contador`. Utiliza tu candado para 4 hilos para tener consistencia en tu `Contador`.

   Respuesta:

   `ExecuteBakery4.java`:

   Igual que en el Ejercicio 1, ejecuta 400 tareas en un `ExecutorService` de 4 hilos sobre un `CounterNaive`, asignando a cada uno de los 4 hilos reales del pool un id fijo (0 a 3) vía `ThreadLocal`, usado en todos los llamados a `Bakery4`.

   (b) De alguna forma obtén el número de veces que los hilos aumentan el contador. ¿Cada uno realiza exactamente 100 tareas o hay algunos que realizan más?

   Respuesta:

   No, cada hilo no realiza exactamente 100 tareas, pero el reparto es notablemente más parejo que con `DoblePeterson` del Ejercicio 1:

```
Hilo id 0 ejecuto 81 tareas
Hilo id 1 ejecuto 224 tareas
Hilo id 2 ejecuto 94 tareas
Hilo id 3 ejecuto 1 tareas
PS C:\Users\Vianney\Desktop\practica3>
PS C:\Users\Vianney\Desktop\practica3> java unam.fc.concurrent.practica3.ExecuteBakery4
Contador final: 400
Hilo id 0 ejecuto 158 tareas
Hilo id 1 ejecuto 94 tareas
Hilo id 2 ejecuto 64 tareas
Hilo id 3 ejecuto 84 tareas
PS C:\Users\Vianney\Desktop\practica3> java unam.fc.concurrent.practica3.ExecuteBakery4
Contador final: 400
Hilo id 0 ejecuto 39 tareas
Hilo id 1 ejecuto 132 tareas
Hilo id 2 ejecuto 169 tareas
Hilo id 3 ejecuto 60 tareas
PS C:\Users\Vianney\Desktop\practica3> java unam.fc.concurrent.practica3.ExecuteBakery4
Contador final: 400
Hilo id 0 ejecuto 23 tareas
Hilo id 1 ejecuto 155 tareas
Hilo id 2 ejecuto 183 tareas
Hilo id 3 ejecuto 39 tareas
PS C:\Users\Vianney\Desktop\practica3> java unam.fc.concurrent.practica3.ExecuteBakery4
Contador final: 400
Hilo id 0 ejecuto 109 tareas
Hilo id 1 ejecuto 109 tareas
Hilo id 2 ejecuto 75 tareas
Hilo id 3 ejecuto 107 tareas
PS C:\Users\Vianney\Desktop\practica3>
```

A diferencia del Ejercicio 1, aquí no hay un patrón sistemático de un mismo par de hilos acaparando la carga; en 4 de las 5 ejecuciones los cuatro hilos alcanzan al menos varias decenas de tareas cada uno, y en la Ejecución 5 el reparto es casi equitativo (109, 109, 75, 107).

(c) ¿Consideras que la implementación cumple con **Justicia**? Justifica tu respuesta.

Respuesta:

Tampoco cumple con Justicia en el sentido estricto de reparto exactamente equitativo, pero se acerca más que el candado del Ejercicio 1. Esto tiene sentido con el diseño del algoritmo: Bakery aproxima un orden FCFS (First-Come-First-Served) global mediante el número de `label`, y los 4 hilos compiten directamente entre sí sobre el mismo arreglo compartido, sin la estructura de árbol de `DoblePeterson` que le da una ventaja estructural fija al par {0,1} sobre el par {2,3}. Por eso en Bakery no se observó el sesgo sistemático hacia los mismos dos hilos que sí se vio en el Ejercicio 1.

Aun así, no es Justicia perfecta: como el número de `label` se calcula leyendo `label` de los demás sin sincronización atómica, dos hilos pueden sacar el mismo número o números muy cercanos si piden el candado casi al mismo tiempo, y las diferencias de scheduling del sistema operativo (qué hilo real arranca antes, cuál obtiene más tiempo de CPU) siguen influyendo en cuántas veces cada hilo alcanza a repetir su ciclo de `lock()`/`unlock()` durante los 400 incrementos.

3. Revisa el programa de Bakery que se les compartió como ejemplo, ejecútalo varias veces, revisa el código y contesta:

(a) Revisa para qué sirve el campo `AtomicReference`, y qué hacen los métodos `compareAndSet()` y `get()`.

Respuesta:

El campo `next` de `Nodo` es un `AtomicReference<Nodo>` en vez de una referencia normal porque varios hilos intentan enlazar un nodo nuevo al final de la lista al mismo tiempo, y sin una operación atómica dos hilos podrían leer el mismo "final de la lista" sobrescribirse uno al otro (perdiendo un nodo).

`compareAndSet(esperado, nuevo)` es la operación CAS (*compare-and-swap*): compara atómicamente el valor actual de la referencia contra `esperado`, y solo si coinciden lo reemplaza por `nuevo`, regresando `true`; si no coinciden (porque otro hilo ya lo cambió), no hace nada y regresa `false`. Es atómica a nivel de hardware, por lo que no necesita candados.

`get()` simplemente lee de forma atómica el valor actual de la referencia.

(b) Describe cómo funciona en a lo más 6 líneas de computadora.

Respuesta:

Cada hilo que quiere el candado crea su propio `Nodo` con `flag = true` e intenta, usando `compareAndSet()`, engancharlo al final de la lista enlazada (a `head.next` si la lista está vacía, o al `next` del último nodo encontrado); si el CAS falla porque otro hilo ganó la carrera, vuelve a intentar leyendo el nuevo final. Una vez enlazado, el hilo espera activamente (`while (last.flag == true)`) a que el nodo justo antes que él en la fila (`last`) baje su bandera, es decir, a que su predecesor libere el candado; `unlock()` simplemente pone en `false` la bandera del propio nodo, dejando pasar a quien está justo detrás. Es, en esencia, un candado de cola (*queue lock*) FIFO implementado sin bloqueos (*lock-free*) mediante CAS.

(c) ¿Si `next` no es un `AtomicReference` sigue funcionando?

Hint: Recuerda la Cola concurrente sin candados que implementaste en la Práctica 2.

Respuesta:

No seguiría funcionando correctamente. Si `next` fuera una referencia normal (no atómica), dos hilos podrían leer al mismo tiempo que el final de la lista es el mismo nodo (por ejemplo ambos ven `tail.next.get() == null` o el mismo `last`), y ambos escribirían su propio nodo ahí sin que el otro se entere: uno de los dos nodos se pierde de la lista (queda çolgado", nadie lo enlaza) exactamente como en la Cola concurrente sin candados de la Práctica 2, donde `enq()` concurrentes sin protección podían pisarse el `next` del mismo nodo y perder un elemento. Aquí, perder un nodo en la fila significaría que ese hilo se queda esperando indefinidamente en su `while(last.flag == true)` sobre un `last` que ya nadie va a actualizar, o peor, que dos hilos entren a la sección crítica al mismo tiempo por haberse enlazado sobre el mismo predecesor.

(d) ¿Consideras que mantiene la lógica de la implementación vista en clase/tu implementación del ejercicio anterior? Justifica porqué.

Respuesta:

Comparte el objetivo de Bakery (orden FIFO justo, sin inanición) pero no el mecanismo. La Bakery de arreglos (Ejercicio 2 y la vista en clase) le da a cada hilo un número (`label`) y, para pasar, cada hilo revisa a los $N - 1$ hilos restantes comparando su número contra el de todos ellos; el costo de cada `lock()` crece con el número de hilos. Esta versión de `Bakery.java`, en cambio, arma una fila explícita (lista enlazada) y cada hilo solo espera a su predecesor inmediato, sin importar cuántos hilos haya en total — de ahí que el comentario del código diga Ünbounded": evita el problema de que el número de `label` crezca sin límite (overflow), sustituyéndolo por una cola de tamaño variable. Entonces sí mantiene la idea central de Bakery (haz fila, respeta el orden de llegada, nadie se queda esperando para

siempre), pero la implementación es la de un candado de cola (*queue lock/CLH-style*) construido con CAS, no la comparación de números de la Bakery clásica.